

Автономная некоммерческая организация высшего образования
«Санкт-Петербургский институт экономики и управления»



Кафедра **09.02.07 Информационные системы и программирование**

ПРАКТИЧЕСКАЯ РАБОТА № 1

Дисциплина(модуль)/предмет **МДК 02.02. Инструментальные средства
разработки и отладки программного обеспечения**

Тема: **Анализ требований и первичное проектирование
структуры репозитория для учебного интеграционного проекта**

Студент (ка)/учащийся: **Иванов Егор Михайлович** / _____ /
Ф. И. О. подпись

Группа / Класс **И14-23-9**

Руководитель: **Чудинова А.А.** / _____ /
Должность, степень, Фамилия и инициалы подпись

Дата представления работы «27» сентября 2026 г.

Работа аттестована с оценкой _____
прописью

**Санкт-Петербург
2026 г.**

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1 КРАТКАЯ ХАРАКТЕРИСТИКА ПРОЕКТИРУЕМОЙ СИСТЕМЫ.....	4
2 ВЫБРАННЫЙ УРОВЕНЬ И ЦЕЛИ ИНТЕГРАЦИИ.....	5
2.1 Уровни интеграции.....	5
2.2 Цели интеграции.....	6
3 КЛЮЧЕВЫЕ ИСТОЧНИКИ И ПРИЁМНИКИ ДАННЫХ.....	7
4 СТРУКТУРА РЕПОЗИТОРИЯ ПРОЕКТА.....	9
4.1 Проектные решения.....	9
4.2 Схема-дерево репозитория.....	9
4.3 Назначение основных каталогов.....	10
4.4 Организация командной разработки.....	11
ЗАКЛЮЧЕНИЕ.....	12
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	13

ВВЕДЕНИЕ

Цель работы — на основе технического описания мини-проекта «Система агрегации и интеграции данных интернет-магазина» проанализировать требования к интеграционному модулю и выполнить первичное проектирование структуры его репозитория.

Задачи работы:

- определить уровень (уровни) интеграции проектируемой системы и сформулировать цели интеграции;
- составить перечень ключевых источников и приёмников данных с указанием форматов, протоколов и режимов обмена;
- спроектировать дерево структуры репозитория проекта и пояснить назначение основных каталогов.

Исходные данные: техническое описание мини-проекта, прикреплённое к заданию [2] (разделы: назначение и цель проекта; архитектура и уровни интеграции; источники и приёмники данных; транспортные протоколы и стандарты; структура репозитория).

1 КРАТКАЯ ХАРАКТЕРИСТИКА ПРОЕКТИРУЕМОЙ СИСТЕМЫ

Проектируемая система — программный модуль интеграции, который автоматизирует обмен данными между внутренней учётной системой торгового предприятия (например, складской CRM) и внешней торговой платформой (маркетплейсом или интернет-магазином).

Модуль является промежуточным звеном («посредником») между двумя независимыми системами: он принимает данные из источников, преобразует их к единому внутреннему представлению, проверяет по правилам бизнес-логики и передаёт в системы-приёмники. Согласно техническому описанию, модуль должен обеспечивать:

- автоматическое обновление остатков товаров на торговой площадке;
- передачу статусов заказов и трек-номеров отправок;
- синхронизацию каталога товаров;
- работу в режиме, близком к реальному времени.

2 ВЫБРАННЫЙ УРОВЕНЬ И ЦЕЛИ ИНТЕГРАЦИИ

2.1 Уровни интеграции

В техническом описании определена многоуровневая архитектура интеграции. Для проектируемой системы выбран комбинированный (многоуровневый) подход, при котором задействуются все три уровня, а ведущим является уровень интерфейсов (API). Характеристика уровней приведена в таблице 1.

Уровень интеграции	Что интегрируется	Технологии и форматы	Роль в системе
Уровень данных	Каталог товаров, остатки и цены из складской системы; заказы в БД учётной системы	СУБД (SQL), файлы JSON / XML / CSV, SFTP для пакетного обмена	Импорт/экспорт каталогов и пакетных файлов, запись новых заказов в базу данных
Уровень интерфейсов (API) — ведущий	Обмен сообщениями с внешней торговой площадкой	REST API, HTTPS, JSON, входящие вебхуки	Отправка обновлённых остатков и трек-номеров, приём уведомлений о новых заказах
Уровень бизнес-логики	Правила обработки данных между системами	Сервисный слой модуля интеграции	Валидация входящих заказов, расчёт актуальных остатков, сопоставление номенклатуры

Таблица 1 – Уровни интеграции проектируемой системы

Обоснование выбора. Ведущим выбран уровень API, поскольку внешняя торговая площадка не предоставляет прямого доступа к своей базе данных, а взаимодействие с ней возможно только через опубликованный REST API и вебхуки. Кроме того, требование работы «в режиме, близком к реальному времени» выполнимо именно за счёт событийного обмена (вебхуки) и запросов к API, а не за счёт периодической выгрузки файлов.

Уровень данных необходим, потому что складская система отдаёт остатки и цены в виде пакетного файла выгрузки, а новые заказы должны сохраняться непосредственно в базе данных учётной системы. Уровень бизнес-логики нужен, чтобы в приёмники попадали только проверенные данные: заказ с некорректными данными или на отсутствующий товар не

должен записываться в учётную систему, а на площадку должен передаваться уже рассчитанный остаток, а не «сырые» цифры склада.

2.2 Цели интеграции

Основная цель интеграции — обеспечить бесшовное взаимодействие учётной системы предприятия и внешней торговой площадки без ручного переноса данных. Она раскрывается в следующих частных целях:

- Актуальность остатков. Автоматически передавать на маркетплейс рассчитанные остатки товаров, чтобы исключить продажу отсутствующего товара.
- Автоматический приём заказов. Получать новые заказы через вебхуки, проверять их и записывать в базу данных учётной системы без участия оператора.
- Обратная связь по заказам. Передавать на площадку статусы заказов и трек-номера отправок.
- Единый каталог. Синхронизировать номенклатуру и цены между системами через импорт/экспорт каталога (JSON/XML/CSV).
- Целостность и достоверность данных. Проверять входящие данные на уровне бизнес-логики до записи в приёмники.
- Безопасность обмена. Использовать защищённые протоколы HTTPS и SFTP, хранить параметры доступа отдельно от исходного кода.
- Сопровождаемость. Разделить модуль на независимые компоненты (подключения, обработка, бизнес-логика), чтобы изменения API площадки или формата выгрузки затрагивали только один компонент.

3 КЛЮЧЕВЫЕ ИСТОЧНИКИ И ПРИЁМНИКИ ДАННЫХ

По техническому описанию в системе выделены два источника и два приёмника данных. Их перечень с характеристиками обмена приведён в таблице 2.

Код	Объект	Передаваемые данные	Формат	Протокол / способ	Режим обмена
И1	Файл выгрузки складской системы	Остатки товаров, цены (номенклатура)	CSV / XML	SFTP	Пакетный, по расписанию
И2	Входящие вебхуки интернет-магазина	Уведомления о новых заказах (состав, покупатель, доставка)	JSON	HTTPS (входящий POST-запрос)	Событийный, близкий к реальному времени
П1	Удалённый сервер маркетплейса	Обновлённые остатки, статусы заказов, трек-номера отправлений	JSON	HTTPS, REST API	По событию / после расчёта остатков
П2	База данных учётной системы	Новые (проверенные) заказы	Записи таблиц СУБД	SQL-запросы через драйвер СУБД	Сразу после валидации заказа

Таблица 2 – Источники и приёмники данных

На основе таблицы 2 выделены основные потоки данных, которые проходят через модуль интеграции:

- Поток «Остатки и цены»: И1 → разбор файла (уровень данных) → расчёт актуальных остатков (бизнес-логика) → П1 через REST API.
- Поток «Заказы»: И2 → приём вебхука (уровень API) → валидация заказа (бизнес-логика) → запись в П2.
- Поток «Трек-номера и статусы»: данные учётной системы об отправке заказа → П1 через REST API.

Примечание – В техническом описании трек-номера указаны только как данные, которые отправляются на маркетплейс, а их источник явно не назван. В работе принято допущение, что трек-номера формируются в учётной системе при отгрузке заказа и считываются модулем из её базы данных.

Общая схема взаимодействия источников, модуля интеграции и приёмников показана на рисунке 1. Схема выполнена в редакторе draw.io с

использованием условных графических обозначений ГОСТ 19.701-90 [1]: источник И1 изображён символом «документ», вебхуки И2 и сервер маркетплейса П1 – символом «данные», уровни модуля – символом «процесс», база данных П2 – символом «запоминающее устройство с прямым доступом». Штриховой рамкой выделены границы модуля интеграции.

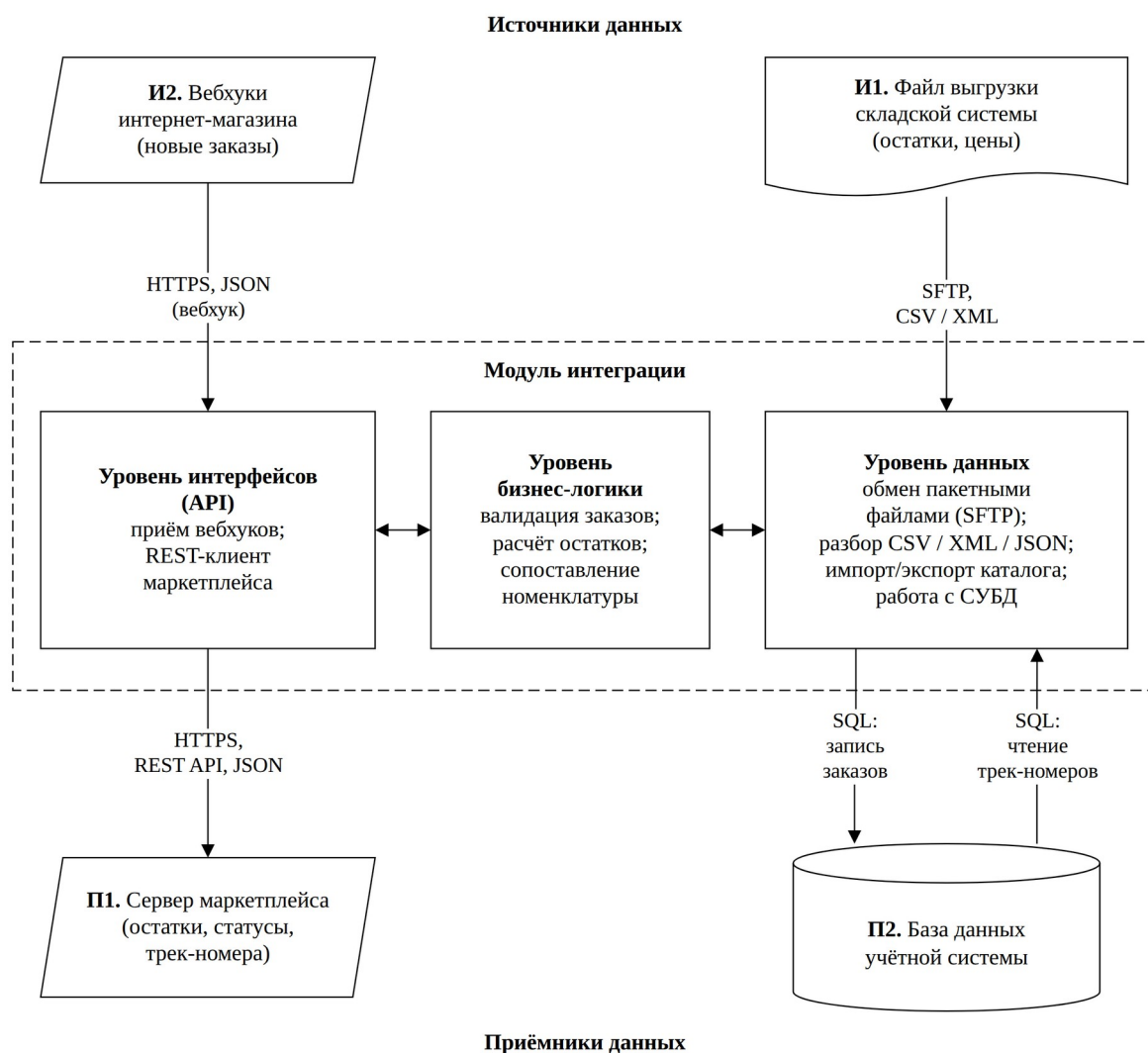


Рисунок 1 – Схема потоков данных системы интеграции

4 СТРУКТУРА РЕПОЗИТОРИЯ ПРОЕКТА

4.1 Проектные решения

Структура репозитория построена на основе обязательных каталогов из технического описания (/src, /config, /tests, /docs) и детализирована с учётом выделенных уровней интеграции. При проектировании приняты следующие решения:

- язык реализации — Python 3 (выбран для учебного проекта: есть готовые библиотеки для HTTP-запросов, SFTP, разбора CSV/XML/JSON и тестирования); структура каталогов при этом не привязана к языку и подходит для других языков;
- внутри /src код разделён по уровням интеграции: подключения к внешним системам (connectors), приём вебхуков (webhooks), разбор форматов (parsers), модели данных (models) и бизнес-логика (services);
- пароли, токены и ключи API не хранятся в репозитории: в него помещаются только шаблоны настроек (*.example), а реальные значения задаются в файле .env, который исключён через .gitignore;
- тестовые данные (примеры файлов выгрузки и вебхуков) хранятся отдельно в tests/fixtures, чтобы проверять интеграционные связи без подключения к реальным системам.

4.2 Схема-дерево репозитория

Планируемая структура репозитория приведена на рисунке 2. Схема выполнена в редакторе draw.io: каталоги изображены в сплошной рамке, файлы – в штриховой; курсивом перечислены файлы, входящие в каталог.

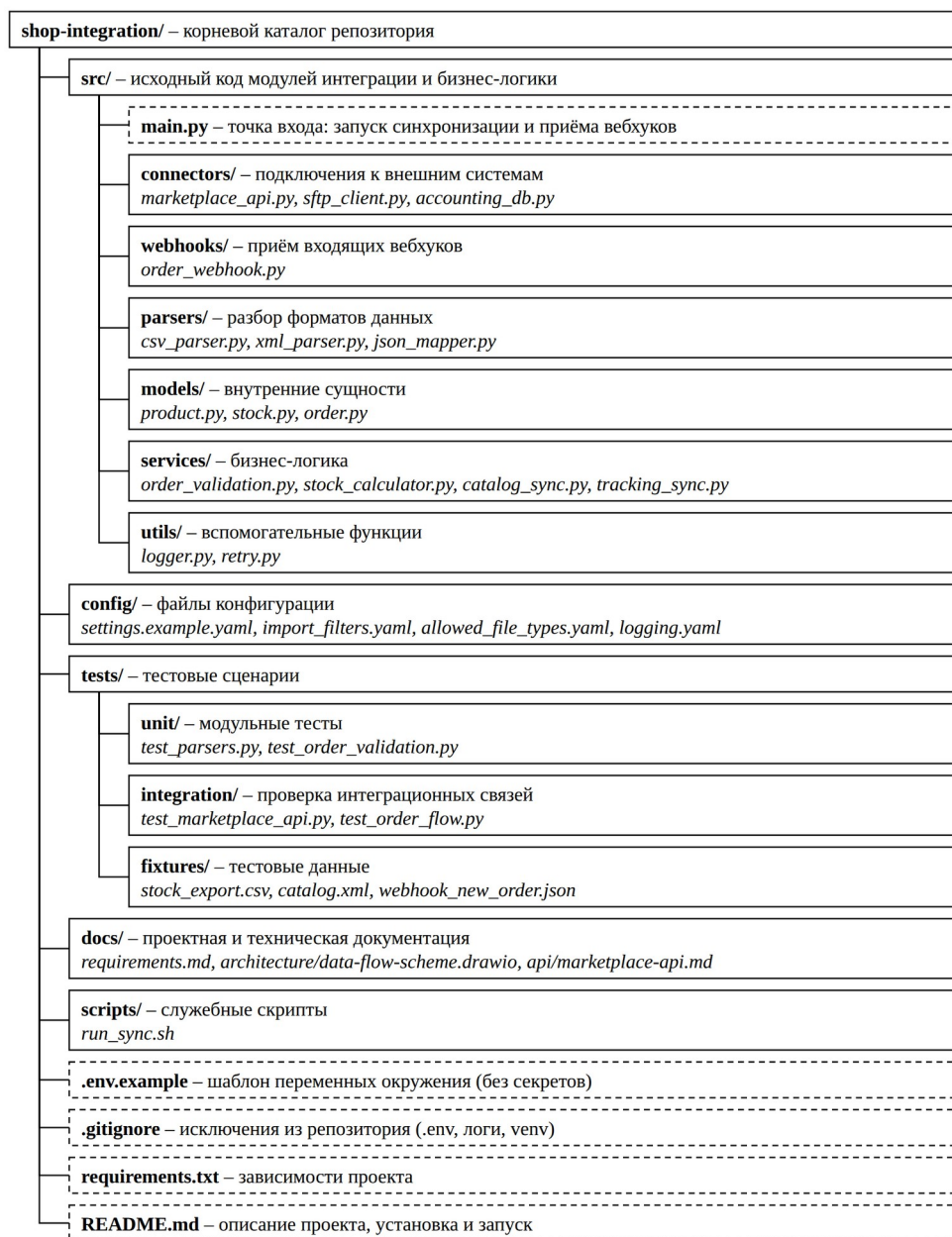


Рисунок 2 – Схема-дерево структуры репозитория проекта

4.3 Назначение основных каталогов

Назначение каталогов и ключевых файлов репозитория описано в таблице 3.

Каталог / файл	Назначение
src/	Исходный код основных модулей интеграции и бизнес-логики. Точка входа — main.py (запуск синхронизации и сервера приёма вебхуков).
src/connectors/	Адаптеры внешних систем (уровень API и доступ к данным): REST-клиент маркетплейса, SFTP-клиент для получения файлов выгрузки, модуль работы с БД учётной системы. Изменение API площадки затрагивает только этот каталог.
src/webhooks/	Приём и первичная проверка входящих вебхуков о новых заказах (проверка подписи и структуры JSON).
src/parsers/	Уровень данных: разбор файлов CSV и XML, преобразование JSON во внутренние модели и обратно.
src/models/	Описание внутренних сущностей: товар, остаток, заказ. Единый формат данных, в который приводятся все источники.
src/services/	Бизнес-логика: валидация заказов, расчёт актуальных остатков, синхронизация каталога, передача трек-номеров.
src/utils/	Вспомогательные функции: журналирование (логи), повтор запросов при сбоях сети.
config/	Файлы конфигурации: параметры подключения к API, пути SFTP, фильтры импорта, перечень разрешённых типов файлов, настройки журналирования.
tests/	Тестовые сценарии: unit — проверка отдельных функций (парсеры, валидация); integration — проверка связей между модулем и внешними системами (с заглушками); fixtures — тестовые файлы и примеры сообщений.
docs/	Проектная и техническая документация: требования, схемы архитектуры и потоков данных, описание используемых методов API.
scripts/	Служебные скрипты для запуска синхронизации вручную или по расписанию.
.env.example	Шаблон переменных окружения (логины, токены, адреса) без реальных значений.
.gitignore	Исключение из репозитория файла .env, логов, временных файлов и виртуального окружения.
requirements.txt	Список зависимостей проекта (библиотеки для HTTP, SFTP, работы с БД, тестирования).
README.md	Описание проекта, инструкция по установке, настройке и запуску, правила работы с репозиторием.

Таблица 3 – Назначение основных каталогов репозитория

4.4 Организация командной разработки

Для командной работы с репозиторием предлагается использовать ветки: main — стабильная версия, develop — текущая разработка, feature/<название> — отдельные задачи (например, feature/stock-sync, feature/order-webhook). Изменения попадают в develop через запрос на слияние (pull request) после прохождения тестов из каталога tests. Такая организация вместе с разделением кода по каталогам позволяет нескольким разработчикам параллельно работать над разными уровнями интеграции без конфликтов.

ЗАКЛЮЧЕНИЕ

В ходе практической работы проанализировано техническое описание мини-проекта «Система агрегации и интеграции данных интернет-магазина». Для системы выбран комбинированный подход к интеграции, включающий уровень данных, уровень интерфейсов (API) и уровень бизнес-логики; ведущим является уровень API, так как обмен с внешней торговой площадкой возможен только через REST API и вебхуки и должен выполняться в режиме, близком к реальному времени. Сформулированы цели интеграции: актуальность остатков, автоматический приём заказов, передача статусов и трек-номеров, синхронизация каталога, достоверность и безопасность обмена.

Определены два источника данных (файл выгрузки складской системы и вебхуки интернет-магазина) и два приёмника (сервер маркетплейса и база данных учётной системы), для каждого указаны формат, протокол и режим обмена, построена схема потоков данных. Спроектирована структура репозитория, в которой обязательные каталоги `/src`, `/config`, `/tests` и `/docs` детализированы по уровням интеграции, а параметры доступа вынесены из исходного кода. Полученная структура является основой для дальнейшей разработки модуля интеграции.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 19.701-90 (ИСО 5807-85). Единая система программной документации. Схемы алгоритмов, программ, данных и систем. Обозначения условные и правила выполнения : межгосударственный стандарт : утвержден и введен в действие Постановлением Государственного комитета СССР по управлению качеством продукции и стандартам от 26 декабря 1990 г. № 3294 : дата введения 1992-01-01. – URL: <https://docs.cntd.ru/document/9041994> (дата обращения: 27.09.2026). – Текст : электронный.